22AIE305 · INTRODUCTION TO CLOUD COMPUTING

JuriScope

AI-Powered Legal Document Assistant on a Cloud-Native RAG Architecture

Zeroth Review Presentation

Group: ____ | Section: AIE ____

Mamidala Som Praneeth Babu · CB.SC.U4AIE24029

THE PROBLEM

Legal Research Is Slow, Expensive, and Inaccessible

- Lawyers and students spend hours manually reading lengthy court judgments to find specific rulings
- Existing tools like Westlaw and LexisNexis are proprietary and expensive — not accessible to students or citizens
- Court PDFs are unstructured — no way to ask a plain-English question and get a direct answer
- No open, cloud-native tool lets you upload any legal document and query it conversationally

20+ min

to manually find one ruling inside a 48-page Supreme Court judgment

The gap: *No open, cloud-native system combines legal document upload, semantic search, and conversational AI in one place.*

LANDSCAPE

Existing Systems Comparison

System	Scope	AI Q&A	Upload Own Docs	Cloud-Native	Open
Westlaw	Full legal DB	No	No	No — proprietary	No
LexisNexis	Full legal DB	Partial	No	No — proprietary	No
ChatGPT	General purpose	Yes	No grounding	No	No
Google NotebookLM	General docs	Yes	Yes	Yes	No
This project	**Legal documents**	**Yes — RAG**	**Yes — PDF/TXT**	**Yes — Azure**	**Yes**

Gap: *No open system combines legal-domain focus, PDF upload, grounded AI answers, and cloud deployment.*

DIRECTION

Objectives & Scope

OBJECTIVES

1. Build a system where users upload legal PDFs and query them in plain English
2. Use Retrieval-Augmented Generation (RAG) to ground every answer in the actual document
3. Store uploaded PDFs in cloud object storage (Azure Blob)
4. Deploy the full system on Microsoft Azure using PaaS services
5. Support session-based multi-user querying with conversation history

SCOPE

- ✦ Accepts PDF and TXT legal documents
- ✦ Answers grounded strictly in indexed documents — no hallucination
- ✦ Two indexed demo documents: Smith v. Johnson (US) and Rajnesh vs. Neha, Supreme Court India (2020)
- ✦ Demonstrator with real documents — not a production legal platform

OUT OF SCOPE

- ✕ Real-time court database integration
- ✕ OCR for scanned / image-based PDFs
- ✕ Mobile application
- ✕ Multi-language support

RATIONALE

Why This Needs a Cloud Architecture

The workload demands it — not a preference, a requirement

Embedding is CPU-heavy

all-mpnet-base-v2 (768D) is slow to run locally on every query

Vector search must scale

Similarity search across thousands of chunks needs a managed index, not a local file

LLM inference needs GPU

Llama 3.3 70B is impossible on a laptop — trivial via the Groq Cloud API

Storage grows indefinitely

PDF storage is unbounded and unsuited to app-server disk

DELIVERY MODEL

PaaS — Azure App Service runs the container, no VM/OS management

DEPLOYMENT MODEL

Public Cloud — Microsoft Azure + external managed APIs

STORAGE MODEL

Azure Blob decoupled from compute — each scales independently

COURSE MAPPING

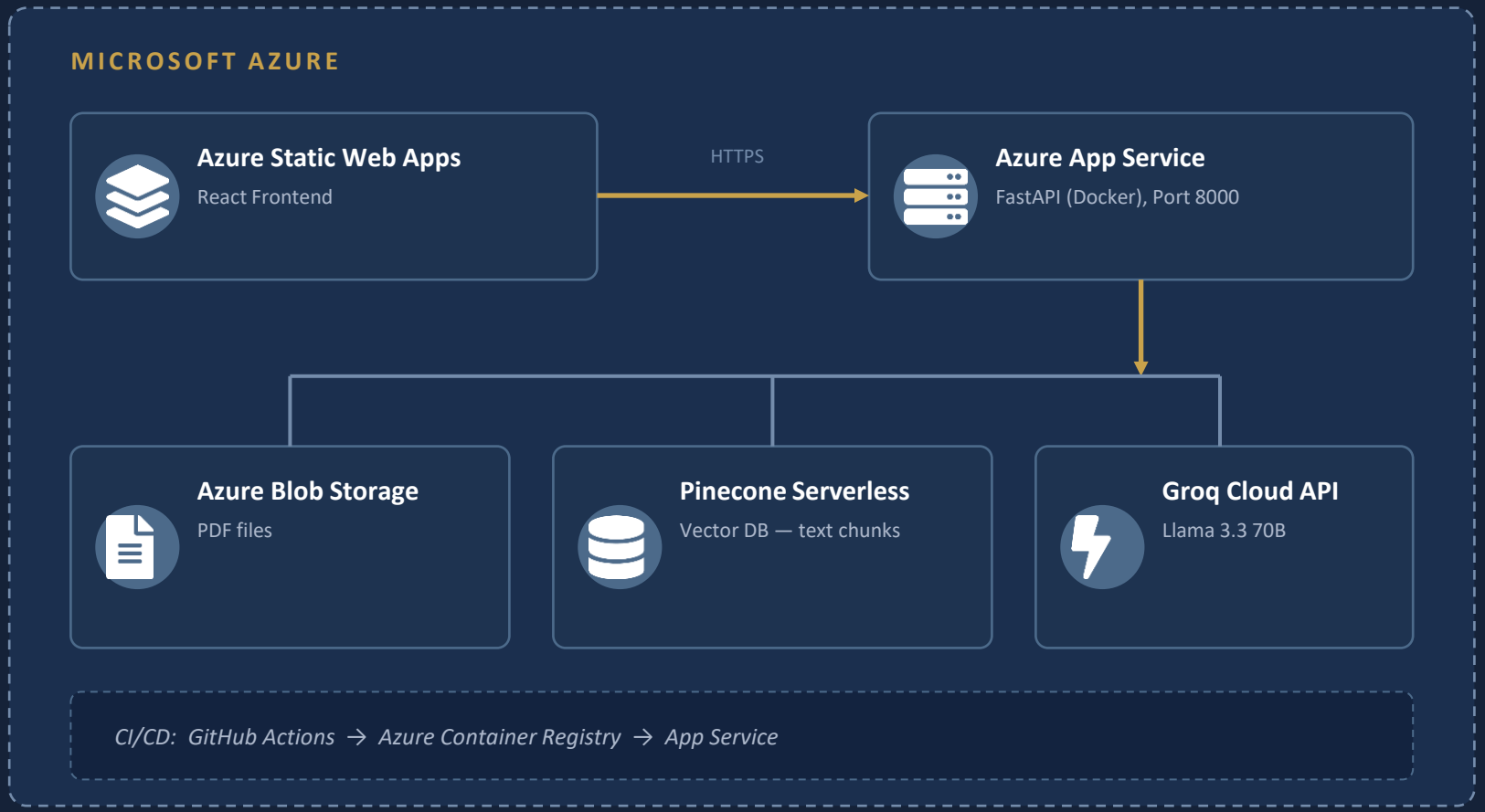
Cloud Concepts Applied

Course Unit	Concept	Applied As
Unit 1	Cloud delivery & deployment models	**PaaS (Azure App Service), Public Cloud**
Unit 2	Cloud compute & object storage	**Azure App Service, Azure Blob Storage**
Unit 3	Containerisation	**Docker, Azure Container Registry**

Why this matters: *each course unit maps to a real, working decision in this project — not a theoretical checklist. The architecture is chosen because the workload needs it, and the course concepts explain why.*

DESIGN

System Architecture



DATA FLOW

1. User uploads PDF → stored in Azure Blob Storage
2. Text extracted → chunked → embedded → stored in Pinecone
3. User asks question → multi-query retrieval from Pinecone
4. Retrieved chunks + question → Groq (Llama 3.3) → answer returned

BUILD

Cloud Services & Tech Stack

Layer	Service	Provider	Status
Frontend hosting	Azure Static Web Apps	Azure	○ Planned
Backend compute	Azure App Service (Docker)	Azure	○ Planned
PDF storage	Azure Blob Storage	Azure	○ Planned
Container registry	Azure Container Registry	Azure	○ Planned
CI/CD	GitHub Actions + ACR webhook	GitHub + Azure	○ Planned
Vector database	Pinecone Serverless (AWS us-east-1)	Pinecone	● Live
LLM inference	Groq Cloud — Llama 3.3 70B	Groq	● Live
Embeddings	HuggingFace all-mpnet-base-v2 (768D)	HuggingFace	● Live
Secrets	Azure App Configuration / .env	Azure	○ Planned

- ● **Live** — running and callable today
- ○ **Planned** — designed and coded for, not yet deployed to Azure

IMPLEMENTATION

Modules Implemented — Backend

Document Upload

Accepts PDF/TXT, extracts text via pypdf, chunks with RecursiveCharacterTextSplitter (1500 chars, 300 overlap), embeds and stores in Pinecone

RAG Pipeline

Multi-query retrieval (3 query variants per question), MMR search (k=6, fetch_k=20), deduplication, context assembly

LLM Answering

Llama 3.3 70B via Groq, grounded strictly in retrieved context, temperature=0.1 for factual accuracy

Session Management

UUID-based sessions, 30-minute timeout, conversation history (last 10 exchanges)

Rate Limiting

20 requests/minute per IP, enforced at the API layer

Input Validation

Query length 3–2000 chars, file size max 20MB, PDF/TXT only

IMPLEMENTATION

Modules Implemented — Frontend

Chat Interface

Real-time Q&A with markdown rendering, typing indicator, auto-scroll

Document Upload Modal

Drag & drop PDF/TXT upload with progress feedback

Indexed Documents Sidebar

Live list of uploaded documents

Backend Health Monitor

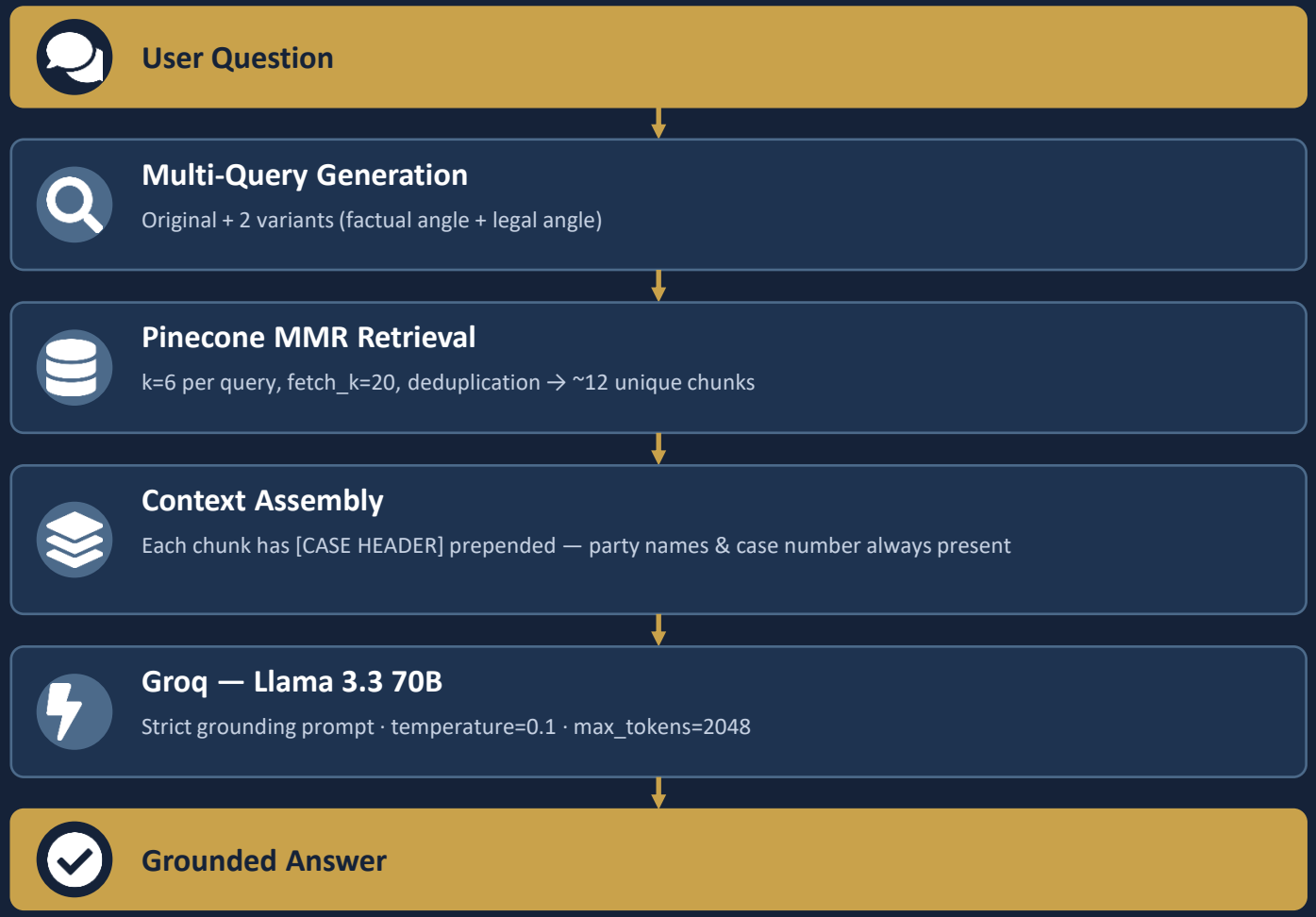
Polls the root endpoint every 5 seconds, shows online/offline status

Session Tracking

Displays the active session ID, supports reset

HOW IT WORKS

RAG Pipeline



KEY DESIGN DECISION

The case header — the first 1200 characters of the document — is prepended to every chunk, so party names and case metadata are always in context regardless of which chunk is retrieved.

WHY MULTI-QUERY?

Legal questions can be phrased factually or in legal terms — generating query variants widens retrieval coverage before deduplicating down to the strongest chunks.

INTERFACE

API Endpoints

Method	Endpoint	Auth	Description
GET	/	None	Health check
POST	/chat	Session	Query documents with RAG
POST	/upload	None	Upload and index PDF/TXT
DELETE	/session/{id}	None	End a session
POST	/session/reset/{id}	None	Reset conversation history
GET	/sessions/count	None	Active session count
GET	/sources/{id}	None	Get session document references

POST /chat — Example

Request

```
{
  "text": "Who are the
   parties in Rajnesh
   vs Neha?",
  "session_id": "abc123"
}
```

Response

```
{
  "response": "The
   parties are Rajnesh
   (appellant/husband)
   and Neha...",
  "session_id": "abc123"
}
```

ENGINEERING

One Codebase, Two Environments

If Env Var Is Set	The App Uses	Otherwise (Local)
`AZURE_STORAGE_CONNECTION_STRING`	Azure Blob Storage for PDFs	Local temp file (deleted after indexing)
`PINECONE_API_KEY`	Pinecone Serverless (cloud)	Required — no local fallback
`GROQ_API_KEY`	Groq Cloud LLM	Required — no local fallback
`PORT`	Dynamic Azure port	Port 8000

WHY IT MATTERS

- Same code, same Docker container — cloud binding is configuration, not a rewrite
- Runs locally with a .env file for development — doesn't burn Azure credits
- Moving to Azure is an App Service environment-variable change, not a code change

RETROSPECTIVE

Challenges & Next Steps

CHALLENGES FACED

LangChain breaking changes — the langchain-community Pinecone integration was sunset mid-project; migrated to the standalone langchain-pinecone package

Package rename — pinecone-client renamed to pinecone; caused import failures until resolved

Cross-case contamination — retriever mixed Smith v. Johnson chunks into Rajnesh vs. Neha answers; fixed with MMR search + case-header injection

Scanned PDF limitation — pypdf cannot extract text from image-based PDFs; documented as a known limitation

Dependency conflicts — sentence-transformers missing from system Python; resolved with an explicit install

NEXT STEPS

1. Deploy backend to Azure App Service via Docker + ACR
2. Deploy frontend to Azure Static Web Apps
3. Add Azure Blob Storage for persistent PDF storage
4. Set up GitHub Actions CI/CD pipeline
5. Add Azure Application Insights for monitoring & telemetry
6. Add OCR support for scanned PDFs via Azure AI Document Intelligence

SOURCES

References

1. Microsoft Azure Documentation — App Service, Blob Storage, Static Web Apps, Container Registry

2. Pinecone Documentation — Serverless Index, Python SDK — pinecone.io/docs

3. Groq Documentation — Llama 3.3 70B Inference API — console.groq.com

4. LangChain Documentation — RAG, Retrieval Chains — python.langchain.com

5. HuggingFace — sentence-transformers/all-mpnet-base-v2 model card

6. Supreme Court of India — Rajnesh vs. Neha and Ors. — MANU/SC/0833/2020 (04.11.2020)

7. Course guidelines — 22AIE305 Introduction to Cloud Computing



Thank You

Juriscope

— Making legal knowledge accessible through cloud-native AI

Mamidala Som Praneeth Babu · CB.SC.U4AIE24029

22AIE305 · Introduction to Cloud Computing